3. Especificação e Modelação do Software

3.1. Apresentação Geral da Especificação e Apoio à Decisão Arquitetural

O projeto de desenvolvimento do sistema integrado de gestão para o Clube Amigos Polvoreira (CAP), uma instituição desportiva sediada em Guimarães [1], exige uma transição rigorosa de procedimentos arcaicos baseados em papel para um ecossistema de software aplicacional centralizado, resiliente e altamente automatizado.[1] A complexidade inerente a este domínio manifesta-se na interseção de múltiplas áreas de negócio: a gestão desportiva de plantéis e convocatórias, a monitorização clínica rigorosa de lesões e exames médicos, o controlo financeiro de quotas com integrações bancárias e a emissão de ficheiros de conformidade fiscal, nomeadamente o ficheiro SAF-T exigido pela Autoridade Tributária.[1] A modelação do software foi dividida em componentes estruturais e comportamentais, garantindo que todas as regras de negócio desportivas, financeiras e clínicas são mapeadas em fluxos de execução determinísticos, inspirando-se nas metodologias canónicas de engenharia de software.[1]
Nesta fase inicial de conceção arquitetural, não se tomou qualquer decisão de forma arbitrária. A definição do alicerce tecnológico de um sistema que lida simultaneamente com dados médicos sensíveis, transações financeiras e operações logísticas de tempo real requer uma análise ponderada e comparativa de múltiplas vias tecnológicas. A integração de conceitos avançados de engenharia, como o isolamento de domínios e a comunicação reativa, moldou o processo de tomada de decisão. As secções que se seguem desconstroem os aspetos críticos da arquitetura global do sistema, apresentando para cada um as opções viáveis, a respetiva análise técnica detalhada e a recomendação final justificada, estabelecendo o guião para a subsequente modelação estrutural e comportamental da aplicação.

3.1.1. Decisão Arquitetural: Padrão Arquitetural Global

A definição do padrão arquitetural dita a forma como os diversos componentes lógicos do sistema comunicam entre si, como os dados são isolados para cumprir imperativos legais e como a aplicação poderá ser escalada no futuro para acomodar o crescimento pretendido pela administração do clube. Foram avaliadas três abordagens arquiteturais distintas para suportar o ecossistema do CAP.
A primeira opção avaliada recaiu sobre a Arquitetura Monolítica em Camadas (Layered Monolith). Esta abordagem, amplamente documentada na literatura clássica de engenharia de software, organiza o código em camadas sobrepostas, tipicamente divididas em apresentação, negócio e acesso a dados.[1] A principal vantagem deste padrão reside na sua simplicidade de implementação e na curva de aprendizagem reduzida para a equipa de desenvolvimento, permitindo um rastreamento linear dos fluxos de execução de ponta a ponta. Além disso,

simplifica drasticamente o processo de implantação (*deployment*) da infraestrutura, visto que toda a aplicação é compilada e distribuída como um único artefacto. Contudo, esta abordagem tende a gerar um elevado acoplamento lógico à medida que o sistema evolui. Num domínio complexo como o do CAP, uma alteração pontual no algoritmo de geração de convocatórias poderia, de forma inadvertida, afetar as validações do módulo financeiro devido à partilha irrestrita de modelos de dados. A ausência de fronteiras estritas entre os domínios dificulta a segregação de dados imposta pelo Regulamento Geral sobre a Proteção de Dados (RGPD), tornando o sistema frágil perante alterações de requisitos.[1]

A segunda alternativa analisada foi a Arquitetura de Microsserviços (Microservices). Este padrão decompõe a aplicação num conjunto de serviços independentes, organizados em torno de capacidades de negócio específicas, que comunicam através de protocolos de rede. A grande vantagem dos microsserviços é a escalabilidade independente e o isolamento físico absoluto. Por exemplo, o módulo responsável pelo processamento de *webhooks* de pagamentos via MBWay e Multibanco [1] poderia ser escalado autonomamente durante os picos de faturação no início de cada mês, sem consumir os recursos dedicados ao módulo de gestão desportiva. Este padrão impõe também um isolamento rigoroso dos dados clínicos, elevando os níveis de segurança. Não obstante, os microsserviços introduzem uma complexidade infraestrutural extrema, exigindo a orquestração de contentores, gestão de falhas de rede e monitorização distribuída. O fator mais penalizador para o contexto do CAP seria a gestão de transações distribuídas. A necessidade de garantir uma consistência forte entre o pagamento de uma quota e a subsequente geração do ficheiro fiscal SAF-T [1] exigiria a implementação de padrões complexos como *Sagas* ou *Two-Phase Commits*, introduzindo um esforço de desenvolvimento e manutenção incomportável face à dimensão da equipa e ao orçamento do projeto.[1]

A terceira via considerada foi o Monólito Modular (Modular Monolith) conjugado com uma Arquitetura Orientada a Eventos (*Event-Driven Architecture*) a nível interno.[2] Esta abordagem retém a simplicidade de um único artefacto de implantação característica do monólito tradicional, mas impõe fronteiras lógicas rigorosas inspiradas nos princípios do *Domain-Driven Design* (DDD). Os diferentes módulos (Clínico, Financeiro, Desportivo e Administrativo) não interagem invocando diretamente os serviços uns dos outros, nem partilham esquemas de base de dados de forma promíscua. Em vez disso, comunicam através de um barramento de eventos (*Event Bus*) em memória. Este mecanismo permite um acoplamento débil altamente responsivo.[2] A título ilustrativo, quando o Exame Médico Desportivo (EMD) de um atleta atinge a caducidade, o módulo clínico emite um evento de domínio interno. O módulo desportivo consome este evento e reage bloqueando a disponibilidade do atleta para convocatórias, sem que exista uma dependência direta em tempo de compilação entre as duas lógicas de negócio.[1] O desafio desta abordagem reside na disciplina arquitetural exigida aos programadores, que devem resistir à tentação de contornar as interfaces dos módulos, uma vez que a barreira de isolamento é lógica e não física.

Opção Arquitetural	**Vantagens**	**Desvantagens**
Monólito em Camadas	Simplicidade de implantação; Curva de aprendizagem	Alto acoplamento lógico; Risco de degradação do código

	reduzida; Facilidade de rastreamento transacional.	(*spaghetti code*); Dificuldade na segregação estrita de dados RGPD.
Microsserviços	Escalabilidade independente por domínio; Isolamento físico total de dados sensíveis; Flexibilidade tecnológica.	Complexidade operacional e infraestrutural elevada; Desafio crítico na gestão de transações distribuídas para o SAF-T; Maior latência.
Monólito Modular	Implantação unificada com baixo acoplamento interno; Consistência transacional garantida; Separação lógica orientada ao domínio (DDD).	Exige elevada disciplina e rigor na manutenção das fronteiras dos módulos; Risco de acoplamento acidental se os princípios não forem respeitados.

Recomendação/Opinião Justificada: A recomendação arquitetural definitiva recai sobre a **Opção 3: Monólito Modular**. Esta decisão é sustentada pelo compromisso ideal entre a necessidade de isolamento de dados clínicos e financeiros (vital para a conformidade com o RGPD e com a Autoridade Tributária) e a capacidade produtiva da equipa de engenharia. A introdução de microsserviços acarretaria um *overhead* operacional injustificado para a atual escala do Clube Amigos Polvoreira.[1] O monólito modular garante o encapsulamento das regras de negócio complexas, permitindo que a plataforma cresça de forma sustentada e que a arquitetura possa, num futuro longínquo e apenas se estritamente necessário, ser desmembrada em microsserviços reais com um esforço de refatorização mínimo.

3.1.2. Decisão Arquitetural: Sistema de Gestão de Base de Dados (SGBD)

A escolha do Sistema de Gestão de Base de Dados constitui a decisão mais crítica para garantir a integridade, a resiliência e a conformidade legal da plataforma do CAP. A base de dados deve suportar a consolidação de contas correntes parentais [1], a persistência estruturada para a emissão de faturas, a gravação de séries temporais de avaliações antropométricas dinâmicas e o registo de queixas físicas não padronizadas.[1]

A primeira abordagem considerada recaiu sobre o ecossistema NoSQL, tendo o MongoDB como expoente representativo. As bases de dados orientadas a documentos oferecem uma flexibilidade extrema de esquema. Esta característica adequa-se de forma excecional ao armazenamento de métricas de performance desportiva e resultados de avaliações físicas, cujas variáveis (peso, altura, índice de massa corporal, resultados de velocidade) podem evoluir constantemente ao longo das épocas sem necessidade de onerosas migrações estruturais na base de dados.[1] Todavia, o paradigma NoSQL apresenta fragilidades conceptuais na execução de transações multi-documento complexas, não garantindo tradicionalmente o rigor absoluto das propriedades ACID (Atomicidade, Consistência,

Isolamento, Durabilidade). O contexto do CAP dita que o subsistema financeiro não admite ambiguidades: a relação relacional estrita entre a entidade Atleta, a Quota pendente e a emissão do Recibo é fulcral para evitar a duplicação de Números de Identificação Fiscal (NIF) e garantir a exportação impecável do ficheiro normativo SAF-T.[1] O modelo de consistência eventual do NoSQL introduziria um risco fiscal inaceitável.

A segunda via escrutinada centrou-se no Microsoft SQL Server, um motor relacional de referência na indústria de software empresarial. O SQL Server providencia um desempenho formidável no processamento de transações pesadas, ferramentas nativas de inteligência empresarial (*Business Intelligence*) adequadas à geração dos relatórios solicitados pela gerência e uma integração ímpar com as tecnologias do ecossistema Microsoft, tipicamente adotadas em unidades curriculares e projetos académicos destas equipas.[1] Apesar da sua robustez técnica inquestionável e do excelente suporte a operações relacionais, esta tecnologia esbarra em constrangimentos de licenciamento e custos operacionais a longo prazo. Para organizações de média dimensão como um clube desportivo local, os custos associados a instâncias de produção de SQL Server podem inflacionar drasticamente as despesas contínuas de infraestrutura (*OpEx*), além de exigir uma pegada de memória superior em ambientes alojados em nuvem.

A terceira alternativa incidiu sobre o PostgreSQL, um sistema de gestão de bases de dados relacional e objeto-relacional de código aberto (*open-source*). O PostgreSQL destaca-se por oferecer uma conformidade ACID rigorosa, essencial para blindar o núcleo administrativo e evitar a quebra de chaves estrangeiras caso um perfil de utilizador transite para um estado de arquivamento lógico (*soft-delete*), preservando o histórico imutável das transações financeiras.[1] A característica mais diferenciadora desta tecnologia, no entanto, é o suporte nativo e altamente otimizado para colunas do tipo JSONB. Esta funcionalidade permite conciliar o melhor de dois mundos: garante a integridade referencial rígida do modelo relacional para identidades, credenciais e finanças, ao mesmo tempo que permite armazenar blocos de dados não esquematizados — como os registos biométricos variáveis da medicina desportiva e os metadados flexíveis dos treinos — numa única coluna transacionável e pesquisável. A principal ressalva prende-se com a necessidade de um planeamento cuidado na criação de índices sobre estruturas JSON para evitar a degradação de desempenho em consultas massivas.

Opção de SGBD	**Vantagens**	**Desvantagens**
MongoDB (NoSQL)	Esquema totalmente dinâmico; Ideal para séries temporais de métricas desportivas e registos clínicos flexíveis.	Suporte limitado para transações ACID complexas; Risco de inconsistência relacional para emissão de ficheiros fiscais (SAF-T).
Microsoft SQL Server	Robustez transacional corporativa; Ferramentas avançadas de auditoria e relatórios; Excelente integração nativa com pilhas tecnológicas.NET.	Elevados custos de licenciamento comercial em ambientes de produção; Maior consumo de recursos infraestruturais.

PostgreSQL	Conformidade ACID estrita a custo zero (*open-source*); Suporte avançado para o tipo JSONB, combinando o modelo relacional com a flexibilidade documental.	A indexação de consultas complexas em campos JSON exige um desenho técnico mais minucioso por parte do administrador de dados.

Recomendação/Opinião Justificada: A adoção do **PostgreSQL** é a decisão arquitetural firmemente recomendada para a infraestrutura de persistência. A capacidade de assegurar uma integridade relacional absoluta resolve categoricamente o imperativo de prevenir anomalias de faturação e duplicação de identidades, salvaguardando a instituição de sanções tributárias.[1] Em simultâneo, o recurso ao tipo de dados JSONB confere a plasticidade estrutural necessária para absorver as particularidades das avaliações antropométricas e dos registos de patologias do módulo clínico [1], eliminando por completo a complexidade operacional de gerir um ecossistema de dados poliglota composto em simultâneo por motores SQL e NoSQL. Esta convergência tecnológica resulta numa arquitetura elegante, segura e economicamente viável para o CAP.

3.1.3. Decisão Arquitetural: Tecnologias e Stack de Backend

O núcleo de processamento do sistema, comummente designado por *backend*, assume a responsabilidade de executar lógicas de negócio pesadas, orquestrar tarefas cíclicas de vigilância e assegurar a criptografia de dados em trânsito e em repouso ao abrigo do RGPD.[1] A seleção tecnológica deve priorizar a velocidade de processamento, a segurança intrínseca e a adequação metodológica.

A primeira hipótese avaliada foi o ecossistema Node.js acompanhado pela linguagem TypeScript, orquestrado pela *framework* NestJS. Esta solução opera sobre um modelo orientado a eventos e chamadas assíncronas assentes num *Event Loop* não bloqueante (*non-blocking I/O*). Trata-se de uma arquitetura fulcral e altamente otimizada para sistemas que lidam com um volume colossal de pedidos de rede concorrentes, sendo particularmente vocacionada para gerir a receção de *webhooks* assíncronos provenientes de *gateways* de pagamento externas (ex: Ifthenpay, SIBS ou Stripe).[1] O NestJS impõe, por definição, uma estrutura modular baseada em decoradores, o que alinharia perfeitamente com a visão do Monólito Modular estabelecida anteriormente. A falha estrutural desta abordagem revela-se no processamento intensivo de Unidade Central de Processamento (CPU). Sendo o Node.js essencialmente *single-threaded*, operações que exigem cálculos matemáticos extensivos ou manipulação massiva de strings — como a compilação, validação e agregação cruzada dos dados para a geração do extenso ficheiro XML do SAF-T [3] — podem bloquear a *thread* principal, degradando severamente os tempos de resposta para todos os outros utilizadores conectados simultaneamente à plataforma.

A segunda opção tecnológica investigada foi a linguagem Java impulsionada pela *framework* Spring Boot. O ecossistema Java representa o padrão da indústria para aplicações corporativas

mission-critical, proporcionando uma resiliência excecional e bibliotecas maduras para a gestão contabilística e financeira. A componente Spring Security oferece nativamente ferramentas sofisticadas para mapear as complexas árvores de hierarquia exigidas pelo modelo de Controlo de Acessos Baseado em Papéis (RBAC), facilitando a implementação de restrições de acesso dinâmicas aos módulos clínicos e desportivos conforme o tipo de utilizador autenticado.[1]
Contudo, esta robustez tem um preço: o ambiente de execução Java (JVM) tende a consumir volumes expressivos de memória de forma basilar (*heavy footprint*) e apresenta tempos de inicialização prolongados. Adicionalmente, a verbosidade inerente à linguagem traduz-se em ciclos de desenvolvimento tendencialmente mais morosos, contrariando a agilidade pretendida para o ritmo académico e industrial de um projeto de implementação incremental.
A terceira arquitetura sondada foi o ecossistema Microsoft.NET 8 com linguagem C#. Esta plataforma destaca-se por um desempenho superlativo de compilação *Just-In-Time* (JIT), otimização de memória substancialmente melhorada nas suas versões mais recentes e uma coesão sem paralelo entre as ferramentas de desenvolvimento e a plataforma de execução. A interligação com o *Entity Framework Core* abstrai grande parte da complexidade de acesso ao PostgreSQL, permitindo a escrita de código de persistência fortemente tipado e resistente a injeções de SQL. Para os desafios específicos do CAP, o ecossistema.NET sobressai devido ao suporte nativo e elegante para serviços de execução prolongada em segundo plano (*BackgroundServices* e *IHostedService*). Esta funcionalidade resolve inequivocamente a necessidade de implementar os *cronjobs* internos que monitorizam, diariamente e de forma silenciosa, a aproximação dos prazos de caducidade dos Exames Médicos Desportivos (notificando os intervenientes a 30 dias do término) e o agravamento de quotas financeiras em atraso [1], tudo isto sem necessidade de orquestradores de tarefas externos (como o Celery ou o RabbitMQ).

Opção de Backend	**Vantagens**	**Desvantagens**
Node.js (NestJS)	Elevado desempenho I/O e processamento concorrente assíncrono; Ideal para manipulação de fluxos contínuos de *webhooks* e notificações WebSocket.	O bloqueio da *thread* principal durante a geração intensiva do ficheiro XML SAF-T pode colapsar a responsividade geral do sistema.
Java (Spring Boot)	Padrão corporativo para aplicações transacionais; Ecossistema extremamente maduro para implementações complexas de segurança (RBAC).	Elevado consumo de memória basilar; Verbosidade e morosidade no ciclo de programação e implantação inicial.
C# (.NET 8)	Desempenho computacional elevado; Integração elegante com o banco de dados; Suporte nativo imbatível para tarefas assíncronas recorrentes	Exige proficiência nas abstrações de injeção de dependência e gestão de tempo de vida de instâncias associadas ao *Entity*

	(*cronjobs* e *workers*).	*Framework Core*.

Recomendação/Opinião Justificada: A recomendação tecnológica formaliza-se em torno do **C# com.NET 8**. A exigência imperativa de contar com mecanismos de monitorização contínua (como o *cronjob* de validação do EMD, que previne infrações ativas dos atletas) [1] exige uma infraestrutura capaz de executar tarefas rotineiras de forma segura, concorrente e nativa, uma área onde os *HostedServices* do.NET não encontram rival de igual simplicidade. Simultaneamente, o processamento multi-tarefa eficiente da plataforma lidará confortavelmente com a exigência computacional de extrair e formatar centenas de faturas para o ficheiro legal SAF-T, garantindo a execução fluida e de alta performance de todas as regras de negócio estipuladas.

3.1.4. Decisão Arquitetural: Comunicação e Especificação da API

O sistema arquitetado não atua de forma isolada; ele opera como um motor agnóstico a interfaces visuais (uma vez que não abordamos o design de UI) e interage com múltiplos clientes, desde portais web administrativos a dispositivos móveis dos atletas, requerendo um canal de comunicação rigorosamente especificado.[6] A camada de interface de programação de aplicações (API) define a fronteira contratual entre os clientes e as lógicas de negócio.
A abordagem GraphQL propõe um paradigma onde o cliente possui autonomia para definir exatamente a estrutura de dados que pretende receber. Esta tecnologia erradica o fenómeno de transferência excessiva de dados (*over-fetching*), constituindo uma solução teórica excelente para a renderização dos intrincados *dashboards* de contabilidade solicitados pela gerência e para os complexos ecrãs de agregação do perfil individual do atleta, que fundem métricas desportivas, financeiras e biométricas numa única vista.[1] A limitação fatal do GraphQL no âmbito deste projeto, contudo, é a extrema dificuldade em implementar restrições granulares de segurança. O modelo rigoroso de proteção de dados sensíveis exige que a visualização de aspetos médicos seja ocultada do departamento financeiro.[1] Num grafo de dados altamente interligado, garantir que um *query* não expõe acidentalmente um histórico de queixas físicas através de uma relação não protegida exige a injeção de autorizações complexas em cada nó (*resolver*), escalando a complexidade do projeto de segurança de forma incomportável.
O protocolo gRPC, suportado por *Protocol Buffers* e comunicação HTTP/2 nativa, oferece a solução de menor latência e maior densidade binária da atualidade. Trata-se de uma tecnologia imbatível para a comunicação inter-processual nos casos em que a arquitetura distribuída em microsserviços se encontra presente, reduzindo o volume da carga útil (*payload*) em ordens de magnitude. O gRPC, porém, é hostil a integrações diretas com os navegadores da *web* tradicionais (exigindo a configuração complexa de *proxies gRPC-web*) e carece da universalidade exigida para consumir serviços de terceiros de forma trivial. Uma vez que o CAP necessitará obrigatoriamente de expor pontos de acesso públicos (*endpoints*) para absorver notificações passivas (*webhooks*) emitidas pelas plataformas bancárias (pagamentos de referências Multibanco) [1], a adoção de um protocolo restritivo como o gRPC inviabilizaria ou dificultaria de sobremaneira estas integrações de importância vital.

A Arquitetura RESTful (*Representational State Transfer*), alinhada com os níveis 2 ou 3 do modelo de maturidade de Richardson, preconiza a modelação da interface em torno dos recursos do domínio (ex: /api/atletas, /api/quotas). A sua ubiquidade constitui a sua maior força. O formato textual da notação de objetos JavaScript (JSON) garante a interoperabilidade imediata com qualquer tipo de cliente, assegurando que o processamento imperativo de submissão de avaliações médicas, geração de inscrições nas federações parceiras (US14) e integração transparente com as *gateways* de pagamento operam sobre o paradigma canónico da Internet.[1] Adicionalmente, a segregação de rotas no paradigma REST permite aplicar o Controlo de Acessos Baseado em Papéis de forma trivial ao nível do controlador da aplicação, bloqueando imediatamente, com códigos HTTP 403 (*Forbidden*), qualquer tentativa não autorizada à raiz do pedido, protegendo eficazmente os dados RGPD.[1]

Paradigma de API	**Vantagens**	**Desvantagens**
GraphQL	Elimina a recolha excessiva ou insuficiente de dados; Excelente para a agregação de dados díspares nos *dashboards* de contabilidade e perfis unificados.	Ocultação seletiva de dados sensíveis e implementação granular do RBAC exigem a injeção de camadas de segurança onerosas e propensas a falhas.
gRPC	Latência mínima e forte tipagem binária; Elevado desempenho em redes com baixa largura de banda e elevada concorrência.	Inflexibilidade extrema para integrar com *webhooks* externos em texto plano (Multibanco); Dificulta o consumo a partir de clientes *web* não otimizados.
RESTful API	Aceitação universal; Clareza na manipulação de recursos; Integração nativa, transparente e isenta de fricção com serviços de pagamentos e federações externas.	Requer um planeamento metódico das rotas para evitar a sobrecarga de múltiplos pedidos ao preencher ecrãs com agregação de muitos recursos lógicos.

Recomendação/Opinião Justificada: A estandardização inerente à **RESTful API** consagra esta opção como a recomendação técnica inabalável. Num ecossistema que se encontra dependente da fluidez burocrática e económica, a necessidade incontornável de expor recetores web para assegurar a reconciliação bancária autónoma (US03) elege o REST como o único protocolo viável de forma nativa e sem necessidade de intermediários complexos. A proteção estrita do módulo biomédico e a interdição de fluxos financeiros a atores não-autorizados ficam plenamente asseguradas pelas primitivas de autorização inerentes aos *endpoints* REST.

3.2. Modelação Estrutural

A modelação estrutural define o esqueleto estático do sistema integrado de gestão. É nesta fase que a abstração dos requisitos operacionais documentados se materializa em entidades unívocas, definindo a cardinalidade das relações do domínio, a dependência funcional e as fronteiras lógicas dos módulos que constituem o núcleo da plataforma.

3.2.1. Modelo de Domínio

A conceção do modelo de domínio constitui o exercício de engenharia responsável por destilar a complexidade do Clube Amigos Polvoreira numa grelha concetual rígida, ditada pelos princípios do desenho orientado ao domínio. A premissa central de segurança — o Controlo de Acessos Baseado em Papéis — obriga a que todos os intervenientes da plataforma derivem de uma entidade basilar suprema, a entidade Utilizador. Este nó garante que as propriedades transacionais de segurança, como os identificadores críticos e imutáveis de e-mail e Número de Identificação Fiscal (NIF), juntamente com a abstração das credenciais criptográficas, são uniformemente tratadas.[1]

A entidade Atleta emerge como a herança mais complexa deste ecossistema.[1] Não é apenas um consumidor de acessos, mas sim o vértice onde colidem as restrições logísticas, de saúde e financeiras. Esta entidade acomoda matrizes antropométricas voláteis, e a sua data de nascimento é ativamente invocada na operação contínua de aferição cruzada que dita a elegibilidade desportiva do praticante aquando da alocação estrutural aos Planteis das Equipas.[1]

Numa perspetiva simbiótica, a figura legal do EncarregadoEducacao consubstancia uma entidade dotada de um poderoso relacionamento de cardinalidade de um-para-muitos com a entidade Atleta. A sua existência não é meramente representativa; incorpora a lógica de "perfil parental único" que despoleta uma agregação e consolidação massiva na vertente de pagamentos, unificando a responsabilidade de tesouraria de múltiplos dependentes sob o alçado de uma única conta corrente passível de liquidação aglomerada.[1]

No vetor financeiro, a imutabilidade fiscal ganha forma na tríade relacional Quota, Pagamento e Recibo. A entidade Quota exibe um ciclo de vida autónomo, progredindo num gradiente de estados (Pendente, Pago, Em Atraso). A materialização de um pagamento viabiliza a geração de um Recibo em formato digital, que consolida a transação de forma definitiva. Estes dados não são submetidos a operações de eliminação mecânica, protegendo a estabilidade estrutural exigida para a geração impecável dos relatórios e ficheiros normativos SAF-T em contexto de auditorias da Autoridade Tributária.[1]

A componente da medicina desportiva injeta uma robustez de salvaguarda inegociável, encarnada primariamente na entidade ExameMedicoDesportivo (EMD). A ligação desta entidade com o Atleta transcende a de um simples registo. O estado temporal e a data de validade contida no EMD influenciam catastroficamente a modelação do estado global do indivíduo (propriedade *Apto/Inapto*). Se a validade expirar, o estado propagado interdita por completo a possibilidade matemática de estabelecer uma relação funcional entre o Atleta e as instâncias ativas da entidade Convocatoria.[1]

O diagrama que se segue formaliza as entidades, heranças e cardinalidades supramencionadas, clarificando a organização estrutural deste domínio através da sintaxe PlantUML.[1]

Code snippet

```
@startuml
skinparam classAttributeIconSize 0
skinparam monochrome true
skinparam linetype ortho

class Utilizador {
  + Id : UUID
  + NomeCivil : String
  + Email : String
  + NIF : String
  + HashCredenciais : String
  + Papel : TipoPapel
  + EstadoConta : EstadoContaAtiva
  + DataDeRegisto : DateTime
}

class Atleta {
  + DataNascimento : Date
  + PesoAtual : Float
  + AlturaAtual : Float
  + EstadoClinicoGlob : EstadoAptidao
}

class EncarregadoEducacao {
  + ContactoEmergencia : String
}

class Treinador {
  + GrauDeFormacao : String
}

class EquipaMedica {
  + EspecialidadeClinica : String
}

class Equipa {
```

```
  + Identificador : UUID
  + NomeEquipa : String
  + EscalaoCompetitivo : String
  + LimiteIdadeMinima : Int
  + LimiteIdadeMaxima : Int
}

class Convocatoria {
  + Identificador : UUID
  + DataHoraEvento : DateTime
  + LocalOcorrencia : String
  + NomeAdversario : String
}

class InteracaoConvocatoria {
  + EstadoResposta : DisponibilidadeAtleta
  + DataSubmissao : DateTime
}

class ExameMedicoDesportivo {
  + IdDocumento : UUID
  + CaminhoFicheiro : String
  + DataSubmissao : DateTime
  + DataLimiteValidade : Date
  + EstadoValidacao : EstadoEMD
}

class QueixaFisica {
  + ZonaCorpoAfetada : String
  + NivelIntensidadeDor : Int
  + DataDaOcorrencia : DateTime
}

class QuotaMensal {
  + ReferenciaMes : Date
  + MontanteEuros : Float
  + EstadoLiquidacao : EstadoQuota
}

class TransacaoPagamento {
  + MetodoUtilizado : MetodoCobranca
  + CodigoReferencia : String
  + TimestampSucesso : DateTime
```

```
}

class DocumentoFatura {
  + CaminhoPDF : String
  + NumeroSequencial : String
  + DataDaEmissao : DateTime
}

class SessaoTreino {
  + DataInicio : DateTime
  + DataFim : DateTime
  + LocalFisico : String
}

class RegistoPresenca {
  + EstevePresente : Boolean
  + ObservacoesFalta : String
}

Utilizador <|-- Atleta
Utilizador <|-- EncarregadoEducacao
Utilizador <|-- Treinador
Utilizador <|-- EquipaMedica

EncarregadoEducacao "1" -- "0..*" Atleta : gerencia finanças >
Equipa "1" -- "1..*" Atleta : constitui plantel >
Treinador "1..*" -- "1..*" Equipa : alocado a >

Atleta "1" *-- "0..*" ExameMedicoDesportivo : possui histórico de >
Atleta "1" *-- "0..*" QueixaFisica : reporta >

Atleta "1" *-- "0..*" QuotaMensal : possui dívida de >
QuotaMensal "1" -- "0..1" TransacaoPagamento : liquidada através de >
TransacaoPagamento "1" -- "1" DocumentoFatura : origina emissão de >

Equipa "1" *-- "0..*" Convocatoria : organiza >
Convocatoria "1" *-- "0..*" InteracaoConvocatoria : regista lista >
Atleta "1" -- "0..*" InteracaoConvocatoria : declara presença >

Equipa "1" *-- "0..*" SessaoTreino : executa >
SessaoTreino "1" *-- "0..*" RegistoPresenca : agrega listagem >
Atleta "1" -- "0..*" RegistoPresenca : possui registo >
```

@enduml

3.2.2. Arquitetura de Componentes Lógicos

A transposição do modelo de domínio para uma arquitetura aplicacional requer uma estratégia de encapsulamento defensivo. Ao materializar o padrão arquitetural de Monólito Modular escolhido, o diagrama de componentes subsequente descreve a orquestração do sistema global em quatro grandes blocos contíguos de negócio: o Subsistema de Identidade (GestaoUtilizadores), o Subsistema Desportivo (GestaoDesportiva), o Subsistema Financeiro (GestaoFinanceira) e o Subsistema de Monitorização Clínica (GestaoClinica).[1]
Esta organização não é meramente cosmética; garante um princípio de *Dependency Inversion* rigoroso. A camada de apresentação (clientes web interativos ou *endpoints* móveis) comunica exclusivamente através de interfaces bem padronizadas (ICamadaNegocio ou IAPI_Endpoints), isolando as engrenagens de processamento de regras lógicas de qualquer manipulação indevida. A característica arquitetural de maior destaque é o barramento de comunicação assíncrono interno, vulgarmente conhecido como *Event Bus* (implementado através do padrão de desenho Mediator).
A introdução deste barramento permite resolver de forma elegante e resiliente a necessidade de reatividade transversal do sistema.[2] Por exemplo, o Serviço de Background responsável pela injeção temporal de tarefas (o componente *Cronjob Worker*) desperta diariamente para vasculhar o estado de saúde e financeiro do ecossistema. Quando este motor deteta que uma fatura ultrapassou o seu limiar, ele não escreve diretamente na base de dados desportiva do atleta para o suspender; este emite, sim, um evento de domínio assíncrono (ex: AtletaInadimplenteEvento) para o *Event Bus*. O componente de Controlo de Acessos reage a este sinal e aplica a restrição cautelar (bloqueio). Este grau de isolamento é imperativo para certificar que a alteração do fluxo financeiro não desestabiliza a execução da elaboração das convocatórias operacionais.
Adicionalmente, os módulos persistentes garantem isolamento transacional comunicando com esquemas dedicados dentro da base de dados PostgreSQL primária. O módulo de processamento de pagamentos estabelece ligações seguras e expõe portas de escuta (*webhooks*) para a receção não solicitada das confirmações provenientes das entidades de *gateway* financeiras, automatizando o processamento que em outrora consumia centenas de horas à equipa de contabilidade.[1]

Code snippet

```
@startuml
skinparam componentStyle rectangle
skinparam monochrome true

package "Clientes da Plataforma" {
```

```
  as AdminUI
  as AtletaUI
  as StaffUI
}

node "Servidor Aplicacional (Monólito Modular)" {
  [API Gateway & Camada de Autenticação] as Gateway

  as EventBus

  component "Subsistema de Identidade" as IdSystem {
    as AuthCore
    as ProfCore
  }

  component "Subsistema Desportivo" as SportSystem {
    [Motor de Convocatórias e Agendamento] as MatchCore
    as MetricCore
  }

  component "Subsistema Financeiro" as FinSystem {
    as PayCore
    as SaftCore
    as QuotaCore
  }

  component "Subsistema Clínico" as ClinSystem {
    as EmdCore
    [Processador de Queixas Físicas] as MedCore
  }

  component "Serviços Contínuos (Background)" as Background {
    as CronWorker
  }

  component "Motor de Mensageria" as Notifications {
    as Alerter
  }
}

database "PostgreSQL (Base de Dados Central)" as DB {
  [Esquema 'Identidade']
```

```
  [Esquema 'Clinico']
  [Esquema 'Financeiro']
}

cloud "Entidades e Integrações Externas" as Externals {
  as FinancialGateway
  as SMTP
}

AdminUI --> Gateway : Rest/HTTPS (Token JWT)
AtletaUI --> Gateway : Rest/HTTPS (Token JWT)
StaffUI --> Gateway : Rest/HTTPS (Token JWT)

Gateway --> IdSystem
Gateway --> SportSystem
Gateway --> FinSystem
Gateway --> ClinSystem

IdSystem <--> EventBus : "Publica e Consome Eventos"
SportSystem <--> EventBus : "Reage a Mudanças Clínicas/Financeiras"
FinSystem <--> EventBus : "Informa Confirmações de Débito"
ClinSystem <--> EventBus : "Publica Estados de Inaptidão"
Notifications <--> EventBus : "Dispara Mensagens Notificadas"
CronWorker --> EventBus : "Injeta Gatilhos Temporais Diários"

IdSystem --> [Esquema 'Identidade']
SportSystem -->
FinSystem --> [Esquema 'Financeiro']
ClinSystem --> [Esquema 'Clinico']

PayCore <..> FinancialGateway : "Receção de Webhooks Assíncronos"
Alerter --> SMTP : "Expede Comunicações"

@enduml
```

3.3. Modelação Comportamental

A fase comportamental incide sobre a formalização dinâmica e temporal dos processos. Embora o levantamento preliminar dos casos de uso tenha descortinado a intencionalidade sistémica (aqui expressamente omitido por predefinição documental do arranque), a tradução desse intuito para o fluxo imutável de transações dita o verdadeiro escopo do software. Para

esse efeito, utilizam-se representações rigorosas dos estados de transição da informação desportiva, médica e a elaboração lógica do escoamento de processos através de diagramas de atividade e de sequência.[1]

3.3.1. Máquina de Estados: Exame Médico Desportivo (EMD) e Quotas

A formalização do ciclo de vida das entidades reguladoras evita que uma miríade de verificações condicionais (*if/else*) polua as lógicas do controlador desportivo. As duas entidades mais restritivas da operação do CAP obedecem a uma máquina de estados explícita: o Exame Médico e a Quota.[1]
No âmbito médico, a conceção não admite que uma entrada seja processada levianamente. O documento ingressa no sistema de forma passiva, bloqueando arquiteturalmente a disponibilidade do atleta, e ascende unicamente perante intervenção e autorização de um agente humano da secretaria ou da medicina.[1] Uma vez ativo, o seu decréscimo em direção à caducidade é rastreado por rotinas de máquina, emitindo pré-avisos temporizados a trinta dias da exaustão do prazo. O cruzamento da data limite despoleta a derrocada autónoma e sem apelo para o estado inativo, amputando os direitos operacionais de treino ou jogo.[1] A máquina de estados abaixo materializa este percurso normativo inegociável.

Code snippet

```
@startuml
skinparam state {
  BackgroundColor White
  BorderColor Black
}

state "Submissão de Documentos (EMD)" as Docs {
  [*] --> A_Aguardar_Validacao : Atleta submete PDF (US05)

  A_Aguardar_Validacao --> Rejeitado_Formatacao : Secretaria deteta ilegibilidade
  Rejeitado_Formatacao --> A_Aguardar_Validacao : Re-submissão do atleta

  A_Aguardar_Validacao --> EMD_Aprovado_Ativo : Secretaria certifica conformidade e data
limite
}

state "Ciclo de Conformidade e Alertas" as Cycle {
  state EMD_Aprovado_Ativo {
    [*] --> Monitorizacao_Diaria
    Monitorizacao_Diaria --> Em_Alerta_Pre_Caducidade : Cronjob deteta prazo = 30 dias
restantes
```

```
    Em_Alerta_Pre_Caducidade : Disparo Automático: Notificação para Atleta e Departamento Clínico
  }
}

Docs --> Cycle

state "Estado de Inaptidão Desportiva" as Block {
  EMD_Aprovado_Ativo --> Interdicao_Imediata : Cronjob deteta ultrapassagem da Data Validade
  Interdicao_Imediata : Estado Arquitetural: Bloqueio Total
  Interdicao_Imediata : Restrição ativa em módulos de Seleção (US19)
}

Block --> A_Aguardar_Validacao : Submissão de renovação anual

@enduml
```

3.3.2. Diagramas de Atividade: Tarefas Cronometradas e Incidentes Físicos

Os diagramas de atividade revelam a capilaridade das operações de processamento da aplicação, evidenciando o controlo dos fluxos, desde nós de bifurcação a rotinas executadas paralelamente pelo núcleo de computação.

Atividade 1: Agendamento Diário de Triagem Médica e Financeira A execução assíncrona diária é o motor invisível da salvaguarda legal da organização desportiva. A presente atividade detalha o funcionamento rigoroso do *cronjob worker* que assegura a integridade clínica. Inicialmente, o sistema compila o conjunto volumoso de Exames Médicos ativos no repositório persistente.[1] Numa operação intensiva, efetua-se o cálculo matemático diferencial entre o limiar de validade armazenado e o registo do dia corrente.[1]

Este cálculo direciona o fluxo de controlo para ramificações cruciais. Na eventualidade de o limite coincidir rigorosamente com a marca dos trinta dias, o sistema desdobra o seu processamento: invoca, simultaneamente, os despachos de avisos de comunicação para as entidades Atleta e Equipa Médica, num processo paralelo de bifurcação (*fork/join*).[1] Se a diferença temporal atingir níveis negativos (passado o prazo de validade), o sistema aplica a execução destrutiva severa: num ambiente transacional estrito, degrada o estado da entidade do atleta, assegurando uma exclusão inegociável nos formulários da equipa técnica. Este algoritmo desvincula totalmente a obrigação humana da tarefa maçadora e letárgica do rastreamento de papéis e cadernos e previne sanções por parte da associação ou liga.[1]

Code snippet

```
@startuml
skinparam activity {
  BackgroundColor White
  BorderColor Black
}

start
:Disparo Temporal (Workers Background Service às 01:00 AM);
:Recolher listagem universal de EMDs no estado "Aprovado_Ativo";

repeat :Início da iteração por Exame;
  :Cálculo aritmético de intervalo: (Data de Validade - Data do Servidor);

  if (Intervalo == 30 dias?) then (Verdadeiro)
    fork
      :Serviço Mensageria: Emitir SMS/Email Atleta;
    fork again
      :Serviço Mensageria: Injetar notificação Portal Médico;
    end fork
  else (Falso)
    if (Intervalo < 0 dias?) then (Sim, Expirou)
      :Inicia Transação na Base de Dados;
      :Modificar EMD para estado "Caducado";
      :Forçar o Atleta associado para estado clínico "Inapto";
      :Invalidar presença do Atleta em convocações não confirmadas;
      :Submeter Transação (Commit);
      :Lançar evento de "Atleta Suspenso" para o Barramento;
    else (Não, prazo regular)
      :Processamento Omitido;
    endif
  endif

repeat while (Existem EMDs por escrutinar?) is (Sim)
->Conclusão;

:Termina o processo de segurança passiva;
stop
@enduml
```

Atividade 2: Registo Autónomo de Patologias e Restrições A segunda atividade mapeia a reatividade organizacional orientada ao bem-estar do utilizador no âmbito das queixas e lesões de natureza física.[1] O instante inicial parte do atleta submetendo de forma inteiramente

autónoma a sua patologia temporária (identificação de grau de esforço ou nível de dor localizada).[1]
A gravidade do processo reside no paralelismo das comunicações subsequentes. A informação transcende o armazenamento primário e injeta-se numa cascata de envios. Por um lado, alerta os técnicos sobre o esforço fisiológico, originando num processo informático a exibição ostensiva e imperativa de um ícone restritivo visual na aplicação do treinador.[1] Esta arquitetura traduz a US06 numa prevenção de danos de elevada performance técnica, onde o processador garante que o treino seguinte terá em conta a informação sintomática registada no dia corrente.[1]

Code snippet

```
@startuml
skinparam activity {
  BackgroundColor White
  BorderColor Black
}

start
:Atleta acede ao Portal Pessoal de Saúde (US06);
:Seleção da zona fisiológica e quantificação da Escala de Dor;
:Submissão dos dados patológicos pelo Atleta;

:A API processa a segurança (Sanitização e Validação);
:Guarda Registo de Incidência na coleção de Histórico Clínico (Transação DB);

fork
  :Notificar Serviço de *Push*: Alerta Urgente Treinador Principal;
fork again
  :Notificar Serviço de *Email*: Resumo Fisioterapeuta/Clínica;
fork again
  :Cache Layer: Anexar *flag* restritiva ao perfil ativo (Sinalética UI);
end fork

:Operação concluída com sucesso reportado ao Praticante;
stop
@enduml
```

3.3.3. Diagramas de Sequência: Resiliência Transacional

O detalhe dos diagramas de sequência destapa o funcionamento íntimo das comunicações

intercelulares entre objetos da camada lógica, ilustrando o peso da escolha arquitetural do PostgreSQL como garantidor de bloqueios de leitura e escrita e as interfaces restritas entre controladores.[1]

Sequência 1: Triagem de Resposta a Convocações Desportivas Esta sequência clarifica as exigências e os obstáculos para efetivar uma simples resposta de disponibilidade (US01).[1] Quando um pedido HTTP de aceitação é recebido (declaração de "Sim"), a API não se limita a assinalar o evento na base de dados de convocatórias. A execução desvia-se sincronicamente para efetuar um interrogatório ao serviço da matriz médica.[1]

Esta inquirição cruzada ilustra um mecanismo de falha limpa: se o serviço clínico retorquir uma suspensão do atleta baseada nos exames arquivados ou lesões averbadas em momento posterior à elaboração da lista, a submissão é ativamente vetada, o pedido interrompido e devolvido com o código HTTP de desautorização. Ultrapassado o filtro, ocorre o registo transacional, a captura integral e persistente da estampa temporal de milissegundos (*timestamp*) para fins disciplinares e, finalmente, a propagação imediata (e via tecnologia WebSocket, idealmente) que garante que o painel de administração da equipa técnica testemunhe a atualização da confirmação do seu jogador em cenário de tempo real.[1]

Code snippet

```
@startuml
skinparam maxMessageSize 150
skinparam monochrome true

actor Atleta as User
participant "Aplicação/UI Cliente" as Client
participant "Controller API\n(Desportivo)" as SportsAPI
participant "Serviço de Negócio\n(Validação Clínica)" as ClinicalSvc
database "PostgreSQL\n(SGBD ACID)" as DB

User -> Client : Clica "Confirma Presença (Sim)"
Client -> SportsAPI : PUT /api/desportivo/convocatorias/{id_conv}/resposta\n{ estado:
"Confirmado" }
activate SportsAPI

SportsAPI -> ClinicalSvc : RequererAvaliacaoElegibilidade(atletaId)
activate ClinicalSvc

ClinicalSvc -> DB : Executa SELECT EstadoClinico FROM Atleta\nWHERE id = atletaId
activate DB
DB --> ClinicalSvc : Retorna {estado: "Apto", EMD_Valido: true}
deactivate DB
```

```
ClinicalSvc --> SportsAPI : Autorização: true (Processo Válido)
deactivate ClinicalSvc

SportsAPI -> DB : BEGIN TRANSACTION
activate DB

SportsAPI -> DB : UPDATE ConvocatoriaAtleta\nSET resposta = "Sim", data_registo =
NOW()\nWHERE id_conv =...
DB --> SportsAPI : Confirmado (1 linha atualizada)

SportsAPI -> DB : COMMIT
deactivate DB

SportsAPI -> Client : HTTP 200 OK (Confirmação Guardada)
deactivate SportsAPI
Client --> User : Exibe notificação de sucesso e calendário atualizado.

@enduml
```

Sequência 2: Integração de Cobranças Externas (Webhook) e SAF-T A espinha dorsal das operações burocráticas centra-se no ecossistema de geração financeira da Instituição. Historicamente assolada pela conciliação de tabelas bancárias manuais, a automação eleva este fluxo a uma coreografia de extrema complexidade transacional e precisão de registos no SGBD.[1]

Quando uma *Gateway* de pagamento externa (por exemplo, a provedora da referências Multibanco) conclui e capta fisicamente o fundo monetário nos terminais de tesouraria, emite um postulado (*Webhook*) não solicitado ao controlador API do clube.[1] Este tráfego penetra nas portas do CAP e inicia uma execução singular: o bloqueio de edição simultânea (*SELECT FOR UPDATE*). Esta trava exclusiva imobiliza a entidade Quota, prevenindo que uma concorrência espúria de uma ação parental em idêntico milissegundo comprometa a sanidade dos saldos financeiros. Assim que confirmada a condição precária da dívida (Pendente ou Em Atraso), a entidade transita imutavelmente para o estado de conformidade.[1] Segue-se a ativação do motor de formatação digital, expedindo instantaneamente a formatação e geração do recibo padronizado PDF para armazenamento protegido. Consolidado o repositório financeiro passível de incorporação integral no descritivo SAF-T mensal exigido pela AT, a conexão encriptada é terminada emitindo garantias operacionais que satisfazem todos os atores na cadeia de evento.

Code snippet

```
@startuml
```

```
skinparam maxMessageSize 150
skinparam monochrome true

participant "Gateway MBWay/SIBS\n(Entidade Externa)" as SIBS
participant "Controller API\n(Financeiro)" as FinAPI
participant "Motor de PDF &\nAuditoria (SAF-T)" as SaftMotor
database "PostgreSQL\n(SGBD Transacional)" as DB

SIBS -> FinAPI : POST Webhook: /api/financas/pagamentos/confirmar\n{ payload: "Ref=1234,
Valor=X, Sucesso" }
activate FinAPI

FinAPI -> DB : Inicia BEGIN TRANSACTION
activate DB

FinAPI -> DB : SELECT * FROM Quotas\nWHERE Ref = 1234 FOR UPDATE (Lock Linha)
DB --> FinAPI : Devolve Objeto {estado: "Pendente"}

alt Quota com estado "Pago"
   FinAPI -> DB : ROLLBACK (Abortar para previnir duplicação)
   FinAPI --> SIBS : HTTP 200 OK (Transação Ignorada Atempadamente)
else Quota Inadimplente/Pendente
   FinAPI -> DB : UPDATE Quotas SET estado = "Pago", dt_pgto = NOW()
   DB --> FinAPI : Confirma 1 Linha Alterada

   FinAPI -> SaftMotor : SolicitarDocumentoLegal(DadosAtleta, Montante, Data)
   activate SaftMotor
   SaftMotor --> FinAPI : Devolve PDF binário e Hash Fiscal Registada
   deactivate SaftMotor

   FinAPI -> DB : INSERT Into FaturasRecibos (url_pdf, timestamp)

   FinAPI -> DB : COMMIT TRANSACTION
   deactivate DB

   FinAPI --> SIBS : HTTP 200 OK (Reconciliação Exata e Bem Sucedida)
   FinAPI -> FinAPI : Disparar Evento Interno: "Notificar Atleta do Sucesso"
end

deactivate FinAPI
@enduml
```

3.4. Especificação da API (Interface e Integrações)

A materialização arquitetural culmina na abstração funcional das portas lógicas do sistema. Num enquadramento onde a arquitetura de *Backend* deve operar isenta de dependência visual para qualquer *frontend* web e ser permeável aos parceiros associados, estabelece-se a infraestrutura da *RESTful API* do CAP (nível de maturidade de Richardson estrutural). Todas as respostas são garantidas sob objetos baseados no padrão universal de Notação JSON, garantindo uniformidade e facilidade de depuração.[1]

A autenticação assenta inteiramente numa arquitetura sem estado (*stateless*), viabilizada pela injeção da insígnia criptográfica JWT (JSON Web Token) nos cabeçalhos da petição (Authorization: Bearer <token>). O *payload* codificado dos tokens comporta as chaves hierárquicas de controlo e o papéis delegados (*Roles*), sendo intercetados atempadamente pela camada intercalar do Gateway API de proteção (middleware).

Os códigos de estado do protocolo de hipertexto (HTTP Status) informam perfeitamente a tipologia da operação. Respostas benignas assinalam HTTP 200 OK e 201 Created; o contencioso das regras de negócio reflete HTTP 400 Bad Request em erros de lógica ou dados submetidos incompletos, 401 Unauthorized pela recusa de permissões na plataforma, 403 Forbidden como muro da blindagem e intransigência na segregação transversal do RGPD [1], e 404 Not Found perante tentativas de acesso a entidades inexistentes ou arquivadas na *soft-delete*.

A tabela subsequente condensa a estrutura de contratos, ditando a interoperacionalidade padronizada para abranger com total minúcia a totalidade dos subsistemas.

Método HTTP	**URI / Rota Funcional (Endpoint)**	**Domínio**	**Descrição do Pedido Funcional**	**Autorização Base (RBAC)**
POST	/api/auth/autenticar	Identidade	Verifica integridade de credenciais cifradas e expede token criptográfico persistente e válido (JWT).	Acesso Universal
POST	/api/utilizadores/atleta	Identidade	Averbamento do perfil com varrimento para detetar integridade dupla (unicidade do e-mail e restrição absoluta	Administração, Secretaria

			de NIF).[1]	
POST	/api/utilizadores/{id}/consolidacao	Identidade	Consubstancia associação paternal entre os dependentes menores e o perfil aglutinador (Encarregado Educ.).	Administração, Secretaria
GET	/api/desporto/equipa/{id}/convocatorias	Desportivo	Agrega as marcações desportivas ativas. Utiliza filtragem condicional (*query parameters*) baseada nas permissões e titularidade.	Treinador, Atleta
PUT	/api/desporto/convocatorias/{id}/disponibilidade	Desportivo	Injeta averbamento do Atleta à resposta do jogo no seu período balizado, marcando com estampagem de tempo.	Atleta (O Próprio)
POST	/api/desporto/treinos/{id}/assiduidade	Desportivo	Abertura submetida pelo técnico validando estatísticas formativas de presenças vitais nas convocatórias.[1]	Treinador Associado
POST	/api/clinico/emd/upload	Clínico	Transporte seguro para alojamento do atestado clínico digital; sinaliza mudança de perfil transitório inativo ("A Aguardar").[1]	Atleta
PUT	/api/clinico/emd/{id	Clínico	Homologa	Equipa Médica,

	}/validacao		fisicamente a documentação, instila prazo e desperta de imediato os mecanismos automáticos para vigência diária.	Secretaria
POST	/api/clinico/queixas-fisicas	Clínico	Emite de forma reativa queixa estruturada (escala fisiológica e gravidade) acionando ramificações severas a treinadores e clínicas.	Atleta
GET	/api/financas/faturas-pendentes	Financeiro	Escrutínio para recuperação da lista de obrigações vincendas formatando dinamicamente e providenciando vetores de cobrança externos.	Atleta, Enc. de Educação
POST	/api/financas/pagamentos/sibs-webhook	Financeiro	Barreira pública mas estritamente qualificada. Canal exclusivo para a conciliação assíncrona entre o Gateway, Recibo e SAF-T.[1]	Intermediário Financeiro
GET	/api/financas/exportacoes/saft-pt	Financeiro	Inicia recolha analítica intensiva sobre todas as faturas e devoluções para a geração normativa	Direção, Administração

			da tabela oficial do Estado.[3]	

O modelo acima, dotado de restrições e clareza taxonómica, blinda o ecossistema do Clube e viabiliza um ciclo ininterrupto de comunicações bidirecionais e padronizadas com intervenientes globais, solidificando a fundação corporativa desenhada para o Clube de Polvoreira e sustentando firmemente o desenvolvimento aplicacional.

3.5. Validação da Especificação Construída

A elaboração da arquitetura estrutural e comportamental consagra as fundações para um ecossistema complexo, cuja ambição transcende a mera desmaterialização do papel, instituindo uma revolução de eficiência, segurança e legalidade. Avaliada perante as rigorosas diretrizes associadas às normativas internacionais para métricas de qualidade de desenvolvimento de *software* de engenharia moderna, como a família **ISO/IEC 25010** [1], a especificação proposta satisfaz plenamente os parâmetros estabelecidos e excede o enquadramento imposto na recolha de requisitos original do CAP.[1]

Ao optar convictamente pela arquitetura de **Monólito Modular** acoplada transversalmente a um **Barramento de Eventos**, garante-se a premissa incontornável de **Adequação Funcional**. Como detalhado exaustivamente na máquina de estados da gestão clínica do atleta (via expiração imutável do Exame Médico Desportivo) e suportada pela blindagem relacional nas aprovações de faturas contra duplicações nocivas na emissão do ficheiro legal **SAF-T**, os riscos burocráticos associados à incúria humana estão definitivamente erradicados do organograma gerencial.

No vetor intrínseco de **Fiabilidade, Desempenho e Segurança**, a seleção robusta de PostgreSQL na base transacional afiança um paradigma ACID capaz de suportar simultaneamente isolamento perante injeções simultâneas e imprevistas provenientes de plataformas bancárias no pagamento das quotas, ao passo que as valências atípicas das colunas em notação JSONB gerem habilmente a imprevisibilidade de métricas relativas ao desempenho corporal sem comprometer a eficiência estrutural da Base de Dados.

Por fim, a **Manutenibilidade e Escalabilidade** revelam o trunfo mestre da implementação de tecnologia *.NET* conjugada com *Background Workers*. As tarefas penosas, em particular a orquestração noturna infalível do controlo da inaptidão legal dos exames dos atletas [1], ocorrem em total separação das dependências lógicas dos controladores principais do desporto, salvaguardando níveis notáveis de coesão, rastreabilidade técnica e imune aos colapsos inerentes a picos massivos de processamento contabilístico. Conclui-se firmemente que as decisões técnicas documentadas não se limitam a transcrever uma instrução, mas criam as vigas fundacionais vitais para garantir que o Clube Amigos Polvoreira atinja o futuro ambicionado, assegurando o sucesso transacional ininterrupto no prosseguimento prático e operatório de codificação.